

Bravo++ multi-mode

DISCLAIMER

This is a script I developed for personal use in the hopes that others would find it useful and fun. I am distributing it for free personal use and I appreciate feedback, but please don't expect me to provide full-time support on this.

If the script doesn't work for you, you can submit the `Log.txt` file, the configuration file you are using and a description of the problem by creating a [GitHub issue](#) or send me a PM and I will try to see if I can solve the problem, but it may take time. I am running X-Plane on both Windows 11 and Linux Mint Cinnamon, and will try to test both platforms as extensively as possible before each release. For Mac OS users, I hope to find willing volunteers to test the script and provide me feedback if something doesn't work, but since I do not have Mac OS myself, I will be limited on how much support I can provide for the platform.

Description

Bravo++ allows you to configure multi-mode functionality, so that you get more out of your Honeycomb Bravo than just the basic autopilot. The default mode (AUTO) will retain the standard autopilot functionality (it can also be overridden), but you can configure additional modes so that you can use the selector switch, buttons and rotating knobs to control other functionality in the aircraft. There are some configuration files provided for the default aircraft such as the Cessna 172, the King Air C90B, and the Cirrus SF50 along with configurations for the Aerobask DA42 and DA62. Hopefully these will be enough so that you can configure your own aircraft and perhaps submit it to the collection.

Prerequisites:

- X-Plane 12
- FlyWithLua NG
- [DataRefTool](#) or [DataRefEditor](#) plugin (if you want to customize or write your own configuration)

The functionality is provided as a FlyWithLua script and consists of 3 parts:

- `BravoMultiMode.lua` - This is the main script that provides all the functionality and is placed in the `FlyWithLua/Scripts` directory.
- `log.lua` - This is a log utility that is used by `BravoMultiMode.lua` and is located in the `FlyWithLua/Modules` directory.
- `preferences.cfg` - A global configuration file located in `FlyWithLua/Modules/bravo++/` that provides default values for thresholds and trim settings across all aircraft. This file can be edited to suit your hardware and personal preference. Aircraft-specific config files override these defaults.
- `config file` - The file can be called either `bravo_multi-mode.cfg`, `bravo_multi-mode.<aircraft name>.cfg`, or `bravo_multi-mode.<aircraft name>.<variant>.cfg` (e.g., `bravo_multi-mode.C90B.EVO.cfg`) and is where you configure all the different modes you want to have on your specific aircraft. Some example files are included in the `FlyWithLua/Modules/bravo++/conf` directory and should be placed directly under the corresponding aircraft folder.

There is also an extra utility called `ButtonLogUtility.lua` that is used to determine which buttons the selector knob is mapped to in X-Plane. By default it is set to 0 and will use the HID to determine the state of the left selector knob, but this will introduce lag (at least on Windows 11). So to have a more responsive update to the GUI it is better to determine the button number X-Plane has assigned to the selector knob when it is set to "alt".

Installation

You should begin by installing the [FlyWithLua](#) plugin for X-Plane 12. If you already have it installed make sure that it is the NG version.

Next you can either download the Bravo++ zip archive from the X-Plane forum or get the latest [release](#) from the GitHub repository. All relevant files are found under the FlyWithLua directory and the entire FlyWithLua directory should be copied under the `plugins` folder.

Configuration

Configuring the buttons in X-Plane

Next you need to configure the Honeycomb Bravo buttons to use Bravo++. You may want to create a base profile (called Bravo++) that X-Plane uses, since this can be reused between aircraft configurations. Otherwise chose an existing profile and start configuring the buttons.

Here are the descriptions you should look for when configuring each button with their corresponding dataref:

- HDG = Bravo++ toggles HDG button (FlyWithLua/Bravo++/hdg_button)
- NAV = Bravo++ toggles NAV button (FlyWithLua/Bravo++/nav_button)
- APR = Bravo++ toggles APR button (FlyWithLua/Bravo++/apr_button)
- REV = Bravo++ toggles REV button (FlyWithLua/Bravo++/rev_button)
- ALT = Bravo++ toggles ALT button (FlyWithLua/Bravo++/alt_button)
- VS = Bravo++ toggles VS button (FlyWithLua/Bravo++/vs_button)
- IAS = Bravo++ toggles IAS button (FlyWithLua/Bravo++/ias_button)
- AUTOPILOT = Bravo++ toggles AUTOPILOT button (FlyWithLua/Bravo++/autopilot_button)

For finding the corresponding command in X-Plane just search for "Bravo++" and you should see all the available options you can map to.

There are also datarefs that are used for toggling/scrolling through the modes. I would suggest having it on a button accessible to the hand that is not used for the Honeycomb Bravo (left hand for most people) on the joystick or yoke.

- For toggling through the modes in one direction you should map it to the command with description `Bravo++ toggles MODE`. So all you do is click on the button and it will move over one mode.
- If you prefer scrolling through the modes with the right rotary encoded knob located on the Honeycomb Bravo, then you need to map the following command with description `Bravo++ activates the mode select when button is held in`. The way this works is that you need to keep the button pressed down while you scroll with the right knob. When you are done you release the button.
- If you don't like these options, you can also map the commands that move the selection up or down to any key or button you like using the provided commands with description `Bravo++ cycle mode up` and `Bravo++ cycle mode down`

A small note on button behavior and assuming:

- `LONG_CLICK_THRESHOLD` = 0.750 (default from preferences.cfg)

- `CONTINUOUS_PRESS_THRESHOLD = 2.0` (default from `preferences.cfg`)

These defaults can be changed in the global `preferences.cfg` file or overridden per-aircraft in your config file.

A click (below 750 msec) will actuate the button or switch and the arrows will stay green during this time. A long click (between 750 - 2000 msec) is used for switches and will change the direction in which the switches will be actuated on the following click. In order to help with the timing, the arrow will turn yellow indicating that you can release it to do a long click. Holding a button down for over 2000 msec will assume you want to sustain a switch and is useful for spring-loaded switches that cannot be activated with a simple click. The arrows will turn a magenta color and will remain so until you release the button.

Finally, there are two internal commands that are often assigned to one of the Honeycomb Bravo buttons.

- The `I/O` button (see one of the example configs) is used for switching between the inner or outer scroll knob. The active state is shown in the knob depiction in the Bravo++ window. The dataref is described as `Bravo++ toggles INNER/OUTER mode`.
- For switches a long click will toggle between up and down state. Each button that implements a switch will either have `^^` above or `vv` below the button label indicating what will happen if the button is pressed. This distinguishes it from buttons that just toggle between two states. So by initiating a long click you toggle how the switch will behave. The dataref is described as `Bravo++ toggles UP/DOWN switch mode`.

Configuring the rocker switches in X-Plane

The rocker switches have two Bravo++ commands each; one for the up position and one for down position. There are 7 rocker switches and they are named `switch1`, `switch2`, `switch3`, etc. As mentioned before, just search for "Bravo++" when binding the keys and you will find the 14 commands that need to be bound to the switches.

Here are the descriptions you should look for when configuring each switch with their corresponding dataref:

- `switch1 (position up)` = Bravo++ command for rocker switch1 when it is positioned up (`FlyWithLua/Bravo++/rocker_switch1_up`)
- `switch1 (position down)` = Bravo++ command for rocker switch1 when it is positioned down (`FlyWithLua/Bravo++/rocker_switch1_down`)
- `switch2 (position up)` = Bravo++ command for rocker switch2 when it is positioned up (`FlyWithLua/Bravo++/rocker_switch2_up`)
- `switch2 (position down)` = Bravo++ command for rocker switch2 when it is positioned down (`FlyWithLua/Bravo++/rocker_switch2_down`)
- `switch3 (position up)` = Bravo++ command for rocker switch3 when it is positioned up (`FlyWithLua/Bravo++/rocker_switch3_up`)
- `switch3 (position down)` = Bravo++ command for rocker switch3 when it is positioned down (`FlyWithLua/Bravo++/rocker_switch3_down`)
- `switch4 (position up)` = Bravo++ command for rocker switch4 when it is positioned up (`FlyWithLua/Bravo++/rocker_switch4_up`)
- `switch4 (position down)` = Bravo++ command for rocker switch4 when it is positioned down (`FlyWithLua/Bravo++/rocker_switch4_down`)
- `switch5 (position up)` = Bravo++ command for rocker switch5 when it is positioned up (`FlyWithLua/Bravo++/rocker_switch5_up`)
- `switch5 (position down)` = Bravo++ command for rocker switch5 when it is positioned down (`FlyWithLua/Bravo++/rocker_switch5_down`)
- `switch6 (position up)` = Bravo++ command for rocker switch6 when it is positioned up (`FlyWithLua/Bravo++/rocker_switch6_up`)
-

- switch6 (position down) = Bravo++ command for rocker switch6 when it is positioned down (FlyWithLua/Bravo++/rocker_switch6_down)
- switch7 (position up) = Bravo++ command for rocker switch7 when it is positioned up (FlyWithLua/Bravo++/rocker_switch7_up)
- switch7 (position down) = Bravo++ command for rocker switch7 when it is positioned down (FlyWithLua/Bravo++/rocker_switch7_down)

Configuring the right rotary encoded knob and the trim wheel

Right rotary encoded knob and the trim wheel - compatibility requirements

The script includes improved HID/decoder handling for the right rotary encoded knob and trim wheel. To ensure reliable behaviour on Windows, you must disable the Honeycomb Configurator (if installed) to keep it from capturing those inputs, otherwise the two input sources will conflict and produce incorrect behaviour.

Two acceptable setups:

1.
For cases where you want to keep the Honeycomb Configurator for other aircraft that don't use Bravo++, simply disable the Honeycomb Configurator via the plugin manager.
2.
For cases where you use Bravo++ for all aircraft, uninstall Honeycomb Configurator.

Using the built-in decoder

- Make sure the Bravo device is plugged in before starting X-Plane (the script will exit if it cannot find the Bravo HID).
- Do not bind the selector knob, right knob or trim wheel in X-Plane if you want the built-in decoder to handle them.

Global Preferences

The `preferences.cfg` file located at `FlyWithLua/Modules/bravo++/preferences.cfg` provides default values used across all aircraft. You can edit this file to customize settings for your hardware and personal preference:

```
# Time in seconds before a button press is considered a "long click"
LONG_CLICK_THRESHOLD=0.750

# Time in seconds before a held button enters continuous repeat mode
CONTINUOUS_PRESS_THRESHOLD=2.0

# Trim increment per wheel click (range -1 to 1)
TRIM_INCREMENT=0.005

# Multiplier applied when turning the trim wheel quickly
TRIM_BOOST=6
```

These values can be overridden on a per-aircraft basis by adding them to your aircraft-specific config file. The precedence order is:

1. Aircraft config file (highest priority)
2. Global `preferences.cfg`

3. OS-specific defaults in the script (lowest priority, used only if neither of the above specify a value)

Example lines to add to your aircraft config file (if you want explicit values):

```
LONG_CLICK_THRESHOLD=0.350  
CONTINUOUS_PRESS_THRESHOLD=0.750
```

Configuring the aircraft

The easiest way to start is to use one of the predefined G1000 configurations (all aircraft, but the King Air C90B) like the for the Cessna 172. You can find the full list of aircraft configurations [here](#).

So let's configure the Cessna 172 that uses the G1000.

Start by copying the file under `Resources\plugins\FlyWithLua\Modules\bravo++\conf\bravo_multi-mode.Cessna_172SP_G1000.cfg` or download the file from [bravo_multi-mode.Cessna_172SP_G1000.cfg](#) and copy to the `Aircraft\Laminar Research\Cessna 172 SP` directory. Note that the Cessna has 3 .acf files and the configuration contains the name that is in `Cessna_172SP_G1000.acf`. This is how the script knows which configuration to use when it starts up. If you start any of the other two variants that aren't G1000 equipped, the script will just stop, since it can't find a corresponding config file.

Config File Detection Order

The script searches for your aircraft's config file in this order:

1. Exact match: `bravo_multi-mode.<aircraft_name>.cfg` — matches the name from the .acf file (e.g., `C90B.acf` → `bravo_multi-mode.C90B.cfg`)
2. Variant match: `bravo_multi-mode.<aircraft_name>.*.cfg` — for aircraft variants sharing the same .acf name (e.g., `C90B.acf` → `bravo_multi-mode.C90B.EVO.cfg`). If multiple variant files exist, one is selected alphabetically and a warning is logged.
3. Generic fallback: `bravo_multi-mode.cfg` — used if no aircraft-specific file is found

This allows you to keep separate config files for different variants of the same aircraft (e.g., stock C90B vs. C90B EVO) without renaming them, as long as they follow the naming convention.

Once the file is copied, you can load the aircraft and hopefully you will now see the Bravo++ window that contains the current mode and status of the `bravo_multi-mode`

the device will light up and the Bravo++ window will now show the text in white. By pressing the "HDG" again it should make the button inactive again.

To change the mode, you need to click the button you assigned to it. Clicking the button allows you to cycle through the different modes. For the Cessna 172, there are 3 modes (AUTO, PFD and MFD) and it is possible to add additional modes if desired, but that is for another day. If you are curious about additional modes you can look at the DA42 or DA62 configuration which contains an additional mode called "SYS" that allows settings the lights, operating the anti-ice system, ignition and auxiliary pumps. You can basically configure whatever you want, but there are some known limitations which I won't take up here now.

If you select the "MFD" mode you will notice that the text changes for most of the content. If you turn the left selector knob to "ALT" you will notice that in the Bravo++ window it now indicates "COM". On the bottom, you will also notice that the labels for the buttons are now different. Some of the buttons do nothing, while others will perform an action. So from this selection you are able to tune the com radio frequencies using the right rotary encoded knob and the buttons. The "IAS" button on Bravo device now toggles whether the rotary encoded knob controls the inner or outer ring of the knob. So when it is set to outer it will control the MHz values of the frequency (118 - 136 MHz), while inner will control the KHz frequencies. The "VS" button will control which frequency is active by swapping the frequencies. The "ALT" button allows you to switch between COM1 and COM2. Notice that the text for these buttons on the Bravo++ window are blue-green. This indicates that they toggle something without causing the led light to go on. The "REV" button, on the other hand, is dark grey and this indicates that the led light will be activated if pressed. In this case it will unmute the COM2 speaker and cause the led light to go on. So try dialing in an ATIS/AWOS frequency at the airport you are at on COM2 and then unmute it by pressing the "REV" button. You should hear the ATIS/AWOS track.

I suggest you explore the rest of the functionality, especially the "FMS" selection, which allows you to access the flightplan menu and procedure menu without using the mouse.

For more advanced configuration I would suggest looking at King Air C90B configuration file.

File format

Once, I have finalized the configuration file I will write a more formal specification, but for now I will provide a quick run down.

Modes, selector labels and button labels

The first section defines what will be shown in the Bravo++ window. These are modes, selector labels and the button labels.

So for the Cessna 172 we have the following:

```
# List all the modes. AUTO must always be included.
MODES = "AUTO,PFD,MFD,SYS"

# Set up the labels
PFD_SELECTOR_LABELS = "COM,NAV,CRS/BARO,HDG/RNG,FMS"
MFD_SELECTOR_LABELS = "COM,NAV,CRS/BARO,HDG/RNG,FMS"
SYS_SELECTOR_LABELS = "FUEL,,,"
```

There are 3 modes (AUTO,PFD and MFD) and the selector labels (the ones corresponding to the left rotary encoded knob on the Honeycomb Bravo) for each mode is specified by using the mode name and then adding `_SELECTOR_LABELS`. The selector labels should have 5 values. Note that the `AUTO` will use the default selector

names (ALT, VS, HDG, CRS and IAS), but can be overridden if explicitly specified in the config file.

Another useful feature for more complicated aircrafts is using the same mode name appended with "_" and a number. This will allow you to assign more than eight buttons to the same mode. You can see an example of this in the King Air C90B configuration file.

Next we assign the button labels to each selector name.

```
AUTO_ALT_BUTTON_LABELS = "HDG, NAV, APR, REV, ALT, VS, FLC, AP"
AUTO_VS_BUTTON_LABELS  = "HDG, NAV, APR, REV, ALT, VS, FLC, AP"
AUTO_HDG_BUTTON_LABELS = "HDG, NAV, APR, REV, ALT, VS, FLC, AP"
AUTO_CRIS_BUTTON_LABELS = "HDG, NAV, APR, REV, ALT, VS, FLC, AP"
AUTO_IAS_BUTTON_LABELS = "HDG, NAV, APR, REV, ALT, VS, FLC, AP"

PFD_ALT_BUTTON_LABELS = "    ,    , COM1, COM2, 1&2, <->, O/I,    "
PFD_VS_BUTTON_LABELS  = "MKR, DME, NAV1, NAV2, 1&2, <->, O/I,    "
PFD_HDG_BUTTON_LABELS = "    ,    ,    ,    , STD BARO, CRS SYNC, O/I,    "
PFD_CRIS_BUTTON_LABELS = "OBS, CDI, DME, TMR/REF, NRST, HDG SYNC, O/I, ALERTS"
PFD_IAS_BUTTON_LABELS = "-D->, MENU, FPL, PROC, CLR, ENT, O/I,    "

MFD_ALT_BUTTON_LABELS = "    ,    , COM1, COM2, 1&2, <->, O/I,    "
MFD_VS_BUTTON_LABELS  = "MKR, DME, NAV1, NAV2, 1&2, <->, O/I,    "
MFD_HDG_BUTTON_LABELS = "    ,    ,    ,    , STD BARO, CRS SYNC, O/I,    "
MFD_CRIS_BUTTON_LABELS = "SYS,    ,    ,    ,    , HDG SYNC, O/I,    "
MFD_IAS_BUTTON_LABELS = "-D->, MENU, FPL, PROC, CLR, ENT, O/I,    "

SYS_ALT_BUTTON_LABELS = "    ,    ,    ,    ,    ,    ,    , FUEL SHUTOFF,    "
```

A button label is specified by using the mode name + selector name separated by – and ending in _BUTTON_LABELS. Note that the selector name is not the same as the selector label name. Here we have to use the default selector names which are: ALT, VS, HDG, CRS and IAS.

For AUTO, it will use default button labels, but like the selector labels they can be overridden by specifying them in the config file.

Note that even if you do not want to have all the buttons assigned to something you need to assign a blank label as seen in the example above.

Aligning Paired Button Labels

When you have paired buttons with labels of different lengths (e.g., "Left IGN" vs "Right IGN"), the auto-scaler may render them at different font sizes because it optimizes each button independently. To force paired buttons to use the same scale, use underscore (–) characters as invisible padding in the shorter label:

```
SYS_HDG_BUTTON_LABELS = "Left_ IGN", "Right IGN"
```

The underscores are measured during layout (so both labels get identical font scaling and wrapping) but are stripped before rendering. The result is that both buttons display at the same size, with the underscores invisible to the user.

Actions for right rotary encoded knob

The rotary encoded knob on the right of the Honeycomb Bravo is used for incrementing or decrementing values for various components of the cockpit. For the GNS530/GNS430 and the G1000 the rotary encoded knobs have

To specify the dataref and condition for deciding when a button's LED light should be lit you first specify the name: `mode name + selector name (optional) + button name` and ending with `_BUTTON_LED`. Again the `selector name` is optional because there are cases where you want the same led to light (or not) irregardless of the selection on the selector knob. Next you need to specify the dataref that contains the value of the state of whatever the corresponding command triggered, followed by a condition that determines when the LED should be lit. The condition supports comparison operators: `>`, `>=`, `<`, `<=`, `!=`, and `=` (equality). If the condition

which element of the array to compare against the condition. So in this case it will check if the value at index 1 satisfies the condition $=1$. If true, the LED is lit. Looking at line 4 you see the same dataref, but this time it specified index 2 instead of 1. This is because in line 3 we turn on the right engine's auto ignition so we need to check the corresponding value in the array.

On line 5 -6 we see an example of a switch where we specify the dataref for the UP and DOWN command. This will result in a slightly different rendering of the button in the Bravo++ window where you will either see a ^^ above or ^^ below the button. Line 7 shows the Bravo++ dataref for toggling between UP or DOWN when using switches.

Actions for rocker switches and leds

The Honeycomb Bravo has 7 rocker switches that can be assigned 2 dataref commands each. The principles of assigning datarefs is pretty much the same as with the buttons.

Let's look at an example for the DA42:

```
SWITCH1_UP="sim/ice/pitot_heat0_on"
SWITCH1_DOWN="sim/ice/pitot_heat0_off"
SWITCH1_LED = "aerobask/sw_pitot_heat,=1"

SWITCH2_UP="aerobask/eng/master1_up"
SWITCH2_DOWN="aerobask/eng/master1_dn"
SWITCH2_LED = "aerobask/eng/sw_master1,=1"

SWITCH3_UP="aerobask/eng/master2_up"
SWITCH3_DOWN="aerobask/eng/master2_dn"
SWITCH3_LED = "aerobask/eng/sw_master2,=1"

SWITCH4_UP="sim/electrical/battery_1_on"
SWITCH4_DOWN="sim/electrical/battery_1_off"
SWITCH4_LED = "sim/cockpit/electrical/battery_on,=1"

SWITCH5_UP="sim/systems/avionics_on"
SWITCH5_DOWN="sim/systems/avionics_off"
SWITCH5_LED = "aerobask/sw_avionics,=1"

SWITCH6_UP="aerobask/eng/fuel_pump1_on"
SWITCH6_DOWN="aerobask/eng/fuel_pump1_off"
SWITCH6_LED = "sim/cockpit/engine/fuel_pump_on,=1,1"

SWITCH7_UP="aerobask/eng/fuel_pump2_on"
SWITCH7_DOWN="aerobask/eng/fuel_pump2_off"
SWITCH7_LED = "sim/cockpit/engine/fuel_pump_on,=1,2"
```

Each switch is distinguished by a number and then by either appending "_UP", "_DOWN" or "_LED". The first two just specify a command dataref to activate depending on whether the switch is up or down. The third specifies a dataref whose value is evaluated against a condition in order to determine whether the LED for the switch should be lit (condition evaluates to true) or not.

Annunciator and gear leds

The annunciator leds and gear leds are pretty straight forward and hopefully this example from the King Air C90B should be clear enough:

```
GEAR_DEPLOYMENT_LED = "sim/flightmodel2/gear/deploy_ratio,<1"
```

```

MASTER_WARNING_LED = "sim/cockpit2/annunciators/master_warning,>0"
FIRE_WARNING_LED = "sim/cockpit2/annunciators/engine_fires,>0"
OIL_LOW_PRESSURE_LED = "sim/cockpit2/annunciators/oil_pressure_low,>0"
FUEL_LOW_PRESSURE_LED = "sim/cockpit2/annunciators/fuel_pressure_low,>0"
ANTI_ICE_1_LED = "sim/cockpit2/ice/ice_pitot_heat_on_pilot,=1"
ANTI_ICE_2_LED = "sim/cockpit2/ice/ice_pitot_heat_on_copilot,=1"
STARTER_ENGAGED_LED = "sim/cockpit2/engine/actuators/starter_hit,=1"
APU_LED = "sim/cockpit2/electrical/APU_running,=1"

```

```

MASTER_CAUTION_LED = "sim/cockpit2/annunciators/master_caution,>0"
VACUUM_LED = "sim/cockpit2/annunciators/low_vacuum,>0"
HYD_LOW_PRESSURE_LED = "sim/cockpit2/annunciators/hydraulic_pressure,>0"
AUX_FUEL_PUMP_1_LED = "sim/cockpit2/fuel/transfer_pump_left,=1"
AUX_FUEL_PUMP_2_LED = "sim/cockpit2/fuel/transfer_pump_right,=1"
PARKING_BRAKE_LED = "sim/cockpit2/controls/parking_brake_ratio,>0"
VOLTS_LOW_LED = "sim/cockpit2/annunciators/low_voltage,>0"
DOOR_1_LED = "sim/flightmodel2/misc/canopy_open_ratio,>0"
DOOR_2_LED = "sim/flightmodel2/misc/door_open_ratio,>0"
DOOR_3_LED = "sim/cockpit2/annunciators/cabin_door_open,>0"

```

Each LED has its own unique name that matches the corresponding LED on the Honeycomb Bravo. Just like in the previous section, you specify a dataref followed by a condition. The condition supports comparison operators: `>`, `>=`, `<`, `<=`, `!=`, and `=` (equality). If the condition evaluates to true, the LED is lit. A bare number without an operator is treated as equality (e.g., `0` is equivalent to `=0`). The only additional feature we have here is that we can tie several datarefs to the same annunciator LED. This is done by adding a `_#` between the annunciator name and the trailing `_LED`. So this means that if any of these dataref conditions evaluate to true, the LED will light up.

Note that the `GEAR_DEPLOYMENT_LED` will most likely always be this value for retractable gears. For fixed gear aircraft you should not specify the `GEAR_DEPLOYMENT_LED` in the configuration.

The trim wheel

The trim wheel overrides the default values used in X-Plane to better simulate a manual trim wheel. The two parameters that can be set are `TRIM_INCREMENT` and `TRIM_BOOST`. The value of the trim is any value between -1 and 1, so the trim increment specifies how much one "click" of the wheel will change the value of the trim.

The trim boost is a value that is applied to the trim increment if the wheel is turned quickly. So if there are many "clicks" in short succession, the trim increment will be multiplied by the trim boost value.

Default values for both `TRIM_INCREMENT` and `TRIM_BOOST` come from `preferences.cfg` (0.005 and 6 respectively) and can be overridden per-aircraft:

```

TRIM_INCREMENT=0.005
TRIM_BOOST=6

```

Troubleshooting

If the Bravo++ window does not appear or the window has no buttons or the selector values don't change when toggling/scrolling, do the following:

- Open the `log.txt` file under the X-Plane directory and search for "BRAVO++ ERROR"
- If no Bravo++ error is found, look for other possible issues in the `log.txt` file
- If all else fails, send me a PM on the X-Plane forum or [create an issue on GitHub](#) with the `log.txt` and

config file if you are not using one of the examples, and I will do my best to help out.

The most common issues are:

- The Honeycomb Bravo wasn't found; is it plugged in?
- The button number assigned to the `alt_selector_button` is wrong; make sure that you select the ALT in teh selector before noting the number.
- The name is invalid; i.e. you have used the wrong combination of `mode name + selector name + button name`.
- The `dataref` doesn't exist. You will usually see this in the log.

Known bugs

I am aware of some minor annoying bugs:

- The button for switching the com frequency doesn't work all the time. You just have to be persistent and press it multiple times. This doesn't happen with the nav frequency, so I am not sure why it's an issue with the com frequency.